PROJECT-BASED LEARNING

PHASE-II EVALUATION

VitalRoute: A Smart Multi-Specialty Hospital Management and Emergency Routing System

Team ID: DSCPP-III-2026-T120

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T120
Semester: 3rd
Sections: ML1, ML4, E
Domain: Healthcare and Hospital Management Systems

Team Members

1. Nakshatra Agarwal – 2510370215 *(ML1)*
2. Devesh Saini – 2510370251 *(ML4)*
3. Vasav Sharma – 25110224030 *(E)*

Mentor

Ms. Pratibha Waldia

Interactions completed: 5

Phase-I Recap & Feedback Incorporated

Phase-I Status

VitalRoute was approved as a console simulator of hospital patient flow. Phase I delivered the problem study, the frozen feature list, the layered architecture, and a mapping of every syllabus topic to a module. Coding was scheduled to start in Phase II.

Feedback Received

- Move from design to a demonstrable prototype
- Make the benefit of automated routing visible to a user, not only to the system
- Keep the integration inside the two PBL courses

Changes Incorporated

- Navigation added: turn-by-turn directions to the assigned bed
- Routing split into resource selection, then route generation
- Database concepts dropped; persistence uses file structures only

Problem Statement & Finalized Objectives

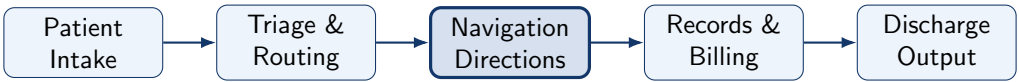
Problem Statement

Patients cannot be treated in arrival order, and once a critical case is stabilised, staff must still find which intensive care bed, operation theatre or ward is free, and how to physically reach it. In smaller hospitals both questions are answered on foot and by telephone, which costs time critical cases cannot afford.

Finalized Objectives

1. Rank emergency patients by clinical severity, and serve routine outpatients in arrival order.
2. Allocate the nearest free resource of the required clinical type by shortest-path search.
3. Generate turn-by-turn directions to that bed, including lift and stairway steps across floors.
4. Keep searchable records and produce an insurance-adjusted bill, with everything persisted to disk.

Updated System Architecture and Workflow



Major Components

- Patient and Staff Management
- Emergency Triage and ER Routing
- In-Hospital Navigation *(new)*
- Consultation and Billing Engine

Changes Since Phase-I

- Navigation added as a fourth application module
- Graph extended with floor numbers, plan coordinates and edge types
- Routing split into resource selection and path synthesis

Module Design & Functional Requirements

Major Modules

1. Patient and Staff Management
2. Emergency Triage and ER Routing
3. In-Hospital Navigation
4. Consultation and Billing Engine

Functional Requirements

- Register a patient and issue an identifier retrievable in constant time
- Order emergency cases by severity, breaking ties by arrival time
- Allocate the nearest free bed of the required clinical type
- Print readable walking directions to that bed
- Accumulate charges automatically and deduct insurance

Module Dependency

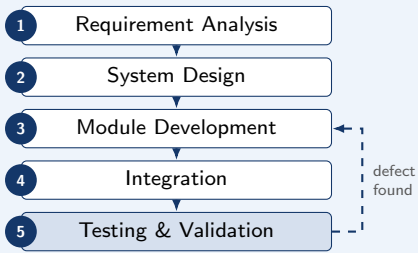
Navigation consumes the hospital graph owned by Routing and runs only after a destination is chosen. Billing reads the record written by Consultation. All four modules share the C engine and one file persistence layer.

Technology Stack & Development Methodology

Technology Stack

- Programming: C for the data-structure engine, C++ for the object layer
- Storage: no external database; file persistence with hashed and AVL indexing
- Framework: C++ Standard Template Library; Qt planned for Phase III
- Development tool: VS Code or Code::Blocks, GCC and G++
- Version control: Git and GitHub

Development Methodology



File structures and indexing belong to the Data Structures syllabus, so persistence is built by hand rather than delegated to a database engine.

Implementation of PBL Course Concepts

Course	Concepts Implemented	Project Module
Data Structures in C	Priority queue, circular queue, double-ended queue, stack, doubly linked list, hashing, AVL tree, weighted graph with shortest path, sorting, binary search	Triage, ER Routing, Navigation, Records
OOPs with C++	Classes and objects, inheritance, virtual functions and polymorphism, operator overloading, exception handling, file handling, STL	Patient and Staff model, Billing Engine, Persistence

Navigation uses the graph twice: Dijkstra selects the shortest route, and the same vertex data then drives the turn-by-turn direction generator.

Development & Implementation Progress

Module	Status	Work Completed
Requirements and architecture	Completed	Feature freeze, four-module design, syllabus mapping
Core data-structure engine	Completed	All queues, stack, AVL and hash index; four unit tests passing
Hospital graph and navigation design	Completed	Floors, coordinates, edge types; bearing and turn classification
Emergency triage	Completed	Severity ordering verified by scenario test
Resource routing	In Progress	Scenario test failing on bed selection; filter fix inside Week 3
Navigation, billing, persistence	Pending	Scheduled for Weeks 4 and 5

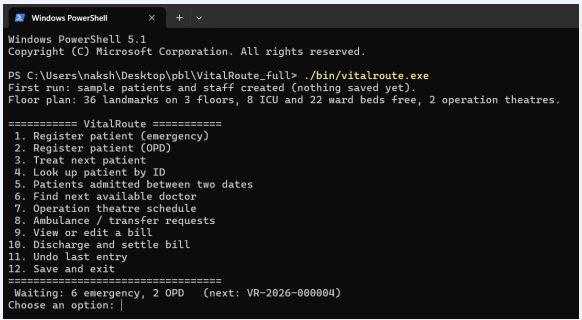
Overall Project Completion: approximately 50%

Working Prototype and Demonstration

Features to Demonstrate

- Emergency intake with severity-ranked treatment order
- Automatic allocation of the nearest free ICU bed, operation theatre or ward
- Turn-by-turn directions printed to the assigned bed
- Running bill with insurance deducted at discharge
- Records reloaded from file after a restart

Prototype Output



First run: sample data loaded, 36 landmarks on 3 floors, 6 emergency and 2 OPD patients waiting.

Testing, Results & Validation

Test or Module	Expected Result	Observed Result	Status
Priority queue	Highest severity first; ties fall back to arrival time	Ordering correct across all cases	Pass
Circular queue	A freed operation theatre slot is reusable without shifting others	Slot reused immediately	Pass
AVL on sorted dates	Height stays logarithmic for ascending admission dates	All four rotations exercised	Pass
Hash lookup	Records retrievable after collisions are forced into one bucket	Retrieval correct	Pass
Routing selection	Nearest free bed of the required type, not merely the nearest bed	ICU patient assigned a nearer ward bed	Fail
Navigation	Expected sequence of turns and floor changes for a known route	Not yet executed	–

Validation Approach: Every structure is unit tested on its own before integration, and all four engine tests pass. The routing scenario test then caught a real defect: distance was compared before candidates were filtered by clinical type, so an intensive care patient was offered a nearer ward bed. The filter now runs first, and the same test is the acceptance check.

GitHub Repository & Development Activity

Repository Information

- GitHub link: `github.com/naksh5116/Team-ID-DSCPP-III-2026-T120`
- README.md: project overview, build steps and module map
- Contributors: all three members added
- Branches: `main`, `feature/engine`, `feature/triage`, `feature/navigation`, `feature/billing`
- Structure: `/src/engine`, `/src/model`, `/src/modules`, `/data`, `/tests`

Development Activity

- Each member commits from their own account on their own feature branch
- Commit messages name the module and the change
- Integration happens through reviewed merges into `main`
- Test files are committed alongside the code they cover

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Individual Contribution

Member	Role	Technical Contribution	Commits
Nakshatra Agarwal	Team Lead	Hospital graph, severity triage on the priority queue, Dijkstra bed selection, direction generator, integration	1
Devesh Saini	Developer	Person, Patient and Staff hierarchy; Charge hierarchy with virtual cost calculation; billing with operator overloading and insurance	1
Vasav Sharma	Developer	Visit records on a doubly linked list, hash index, AVL date-range search, file persistence, test suite	1

Individual Evaluation

Every member is prepared to explain and demonstrate their own technical contribution.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Converting a vertex path into instructions a person can follow
- The nearest free bed is not always the clinically correct one
- Keeping the C engine and the C++ object layer interoperable

Solutions Adopted

- Store coordinates and floor per vertex; classify bearing change into turns; treat lift edges as floor-change steps
- Filter candidates by required resource type before any distance comparison
- Wrap C headers for C++ inclusion and fix ownership rules to avoid double frees

Remaining Work – Weekly Plan to 1 Nov

1. **Wk 1** (30 Sep–5 Oct): core C engine, all queues and stack, with unit tests
2. **Wk 2** (6–12 Oct): C++ class layer, registration, hash index
3. **Wk 3** (13–19 Oct): triage, hospital graph, Dijkstra bed selection
4. **Wk 4** (20–26 Oct): navigation directions, OT scheduling, transfers
5. **Wk 5** (27 Oct–1 Nov): records, billing, persistence, integration and testing

Phase III: Qt graphical interface and role-based reporting, layered on the same unchanged core logic.

Expected Outcomes & References

Expected Final Outcomes

- A complete console simulator covering triage, routing, navigation, records and billing
- Turn-by-turn in-hospital directions produced from the shortest path
- Reproducible, testable results verified against hand-calculated expectations
- Persistent file-based records that survive a restart
- A practical model of hospital operations built only on core data structures and OOP

References

1. Reema Thareja, *Data Structures Using C*, Oxford University Press
2. E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill
3. *Emergency Severity Index Handbook*, Emergency Nurses Association
4. cppreference.com – C++ language and STL reference
5. Qt Documentation, doc.qt.io

Thank You
Questions & Suggestions